0.1 Finite Volume Method

The finite volume method (FVM) is a widely used discretization method for various problems governed by conservation laws. The FVM is particularly robust and well-suited for arbitrary complex geometry. A key advantage of FVM that benefits this simulation is its natural enforcement of local mass conservation.

To begin with, we consider a general hyperbolic conservation law regarding a scalar variable u as

$$\begin{cases} \dfrac{\partial u(\mathbf{x},t)}{\partial t} + \nabla \cdot \mathbf{f}(u;\mathbf{x},t) = 0, \;\; t > 0, \mathbf{x} \in \Omega, \\ u(\mathbf{x},0) = u_0(\mathbf{x}), \qquad\qquad\qquad\qquad \mathbf{x} \in \Omega, \end{cases} \tag{1}$$

where we assume the domain $\Omega \subset \mathbb{R}^2$. The term $\mathbf{f}$ represents the flux transporting the scalar variable u. The basic semi-discretized formulation of the finite volume method can be written as

$$\frac{\partial \overline{u}_E}{\partial t} + \frac{1}{|E|} \int_{\partial E} \mathbf{F}(\mathbf{x},t) \cdot \boldsymbol{\nu} \, d\mathbf{x} = 0, \tag{2}$$

where the cell-averaged value $\overline{u}$ is calculated as

$$\overline{u}_E = \frac{1}{|E|} \int_E u(\mathbf{x},t) \, d\mathbf{x}, \tag{3}$$

where $|E|$ denote the area of the cell E. Here, $\mathbf{F}(\mathbf{x},t)$ denotes the approximated numerical flux. We employ the Lax-Friedrichs numerical flux

$$\mathbf{F}(u^+,u^-) = \frac{1}{2}\Big[\big(\mathbf{f}(u^+) + \mathbf{f}(u^-)\big) \cdot \boldsymbol{\nu} - \alpha_{LF}(u^+ - u^-)\Big], \tag{4}$$

where u^+ and u^- denote the values approximated from the perspective of element $E^+ \in \mathcal{T}_h$ and $E^- \in \mathcal{T}_h$ respectively, where $E^+ \cap E^-$ is an edge of the mesh. The unit normal vector $\boldsymbol{\nu}$ points from E^- to E^+, and is orthogonal to the edge. The parameter α_{LF} is the Lax-Friedrichs stabilizer factor. Depending on the available information about the solution, there are two possible approaches for computing α_{LF}:

- The global Lax-Friedrichs Stabilizer:

$$\alpha_{LF} = \max_{\mathbf{x}\in\Omega,\, t>0,\, u\in\mathbb{R}} \left| \frac{\partial \mathbf{f}(u;\mathbf{x},t) \cdot \boldsymbol{\nu}}{\partial u} \right|;$$

- The local Lax-Friedrichs Stabilizer:

$$\alpha_{LF} = \max_{\text{edges } e} \left(\left| \frac{\partial \mathbf{f}(u^+;\mathbf{x},t) \cdot \boldsymbol{\nu}}{\partial u} \Big|_e \right|, \left| \frac{\partial \mathbf{f}(u^-;\mathbf{x},t) \cdot \boldsymbol{\nu}}{\partial u} \Big|_e \right| \right).$$

The numerical flux $\mathbf{F}$, as defined in equation (4), is consistent and conservative across element interfaces. We now replace the u^+ and u^- with reconstructed values $\mathcal{R}^+$ and $\mathcal{R}^-$ and express the numerical flux as

$$\mathbf{F}(\mathcal{R}^+, \mathcal{R}-) = \frac{1}{2}[(\mathbf{f}(\mathcal{R}^+) + \mathbf{f}(\mathcal{R}^-) \cdot \boldsymbol{\nu} - \alpha_{LF}(\mathcal{R}^+ - \mathcal{R}^-)]. \tag{5}$$

Here, the reconstructed value $\mathcal{R}$ is of higher-order accuracy. We evaluate the integrals over each edge $e_i \in \partial E$ with a suitable quadrature rule. Let $|e_i|$ denote the length of edge e_i. Assuming a quadrature rule with G point per edge, the semi-discretized formulation becomes:

$$\frac{\partial \overline{u}_E}{\partial t} + \sum_{i=0}^{4} \frac{|e_i|}{|E|} \sum_{g=0}^{G} \omega_{i,g} \mathbf{F}(\mathcal{R}^+_{i,g}, \mathcal{R}^-_{i,g}) \cdot \boldsymbol{\nu}_i = 0, \tag{6}$$

where $\omega_{i,g}$ represents the weight associated to the g^{th} quadrature points on edge e_i.

0.2 The Multi-level Weighted Essentially Non-Oscillatory (ML-WENO) Method

We developed an accurate, versatile, robust and efficient finite volume multi level weighted essentially non oscillatory (ML-WENO) reconstruction in Arbogast et al. (2024). ML-WENO is primarily used to reconstruct values on both sides of the element interfaces. It can also be applied to a general reconstruction of discontinuous variables, e.g., piece-wise constant discontinuous Darcy pressure.

0.2.1 Stencil Polynomials

Consider rectangular stencils $\mathcal{S} = \{E_{i,j},\, i \le s,\, j \le t\} \subset \mathcal{T}_h$ on a quasi uniform logically rectangular mesh, which covers the area $\Omega_{\mathcal{S}} = \cup_{E \in \mathcal{S}} E$. For each stencil, we construct a tensor product stencil polynomial $Q^{\mathcal{S}}_{s,t} \in \mathbb{Q}_{s-1,t-1}(\Omega_{\mathcal{S}})$. The scaled tensor product stencil polynomial is expressed as

$$Q^{\mathcal{S}}_{s,t}(x,y) = \sum_{p<s,\ q<t} c_{p,q} \Big(\frac{x - x_{\mathcal{S}}}{h_{\mathcal{S}}}\Big)^p \Big(\frac{y - y_{\mathcal{S}}}{h_{\mathcal{S}}}\Big)^q, \tag{7}$$

where $(x_{\mathcal{S}}, y_{\mathcal{S}})$ is any point in the stencil $\mathcal{S}$. We choose the stencil "center" point: the intersection point of two diagonal lines joining top left, bottom right and top right, bottom left corners. Assuming that each cell $E_{i,j} \in \mathcal{S} \subset \mathcal{T}_h$ is shape regular as in Definition **??**, we define the stencil scale parameter $h_{\mathcal{S}} = \sum_{i \le s, j \le t} E_{i,j}/(st)$ as the average diameter of all cells in the stencil. Introducing the normalized coordinates

$$\xi = \frac{x - x_{\mathcal{S}}}{h_{\mathcal{S}}}, \quad \text{and} \quad \eta = \frac{y - y_{\mathcal{S}}}{h_{\mathcal{S}}}. \tag{8}$$

We can rewrite equation (7) in normalized coordinates as

$$\hat{Q}^{\mathcal{S}}_{s,t}(\xi,\eta) = \sum_{p<s,\ q<t} c_{p,q}\xi^p\eta^q. \tag{9}$$

The tensor product stencil polynomial satisfies the condition that the cell-averaged value (3) is preserved through integration over each cell as

$$\frac{1}{|E_{i,j}|}\iint_{E_{i,j}} Q^{\mathcal{S}}_{s,t}(x,y)\,dxdy = \overline{u}_{i,j}, \quad \forall E_{i,j}\in\mathcal{S}, \tag{10}$$

where $\overline{u}_{E_{i,j}}$ represents the cell-averaged scalar variable over cell $E_{i,j}$.

One way to construct a stencil polynomial $Q^{\mathcal{S}}_{s,t}$ is using basis polynomials. Let $Q^{\mathcal{S}}_{s,t} = \sum_{i,j}\overline{u}_{i,j}\phi^{i,j}_{s,t}$, where the basis polynomials $\phi^{i,j}_{s,t}$ satisfy

$$\frac{1}{|E_{i',j'}|}\iint_{E_{i',j'}} \phi^{i,j}_{s,t}(x,y)\,dxdy = \begin{cases} 1 & (i',j') = (i,j) \\ 0 & (i',j') \neq (i,j) \end{cases}, \quad \forall E_{i',j'}\in\mathcal{S}, \tag{11}$$

which always leads to an $(st)\times(st)$ non-singular linear system. We can write the basis polynomials in terms of normalized coordinates ξ and η as well, and rewrite them as

$$\hat{Q}^{\mathcal{S}}_{s,t}(\xi,\eta) = \sum_{i,j}\overline{u}_k\hat{\phi}^{i,j}_{s,t}(\xi,\eta). \tag{12}$$

The approxmatibility of a tensor product polynomial is related to the associated complete polynomial , as discussed by Dupont and Scott (1980). Given a bounded interpolation projection operator $\mathcal{Q}$, for all $\mathbf{x}\in E\in\mathcal{S}$, we have

$$|D^\alpha u(\mathbf{x}) - D^\alpha\big(\mathcal{Q}u(\mathbf{x})\big)| \leq \begin{cases} Ch^{r-|\alpha|} & \text{if } u \text{ is smooth on the stencil,} \\ C & \text{otherwise,} \end{cases} \tag{13}$$

where $\alpha = (\alpha_1,\alpha_2)$ is a two-dimensional multi-index, and $|\alpha| = \alpha_1+\alpha_2 \leq r$. This theorem applies to our stencil polynomials (7).

In most two-dimensional cases, we implement a square stencil with $s = t = r$. Uneven stencils ($s \neq t$) are used in the following two situations.

- When some a prior knowledge of the solution is acquired before simulation, e.g., the transport velocity is known to be fixed in a single direction.
- When derivatives are approximated, e.g., diffusive flux is computed across the element edge, where a stretched stencil is necessary to compensate for the loss of accuracy associated with derivative approximation.

For simplicity, we restrict the following discussions to the square case $s = t = r$.

0.2.2 Stencil Polynomial Smoothness Indicators

A smoothness indicator σ is defined to quantify the smoothness of the tensor product polynomial $Q^{\mathcal{S}}_{r,r}$ on the stencil $\mathcal{S}$. The classical approach by Jiang and Shu (1996) and Friedrich (1998) defines σ in analogue to a Sobolev seminorm as

$$\sigma_{JS} = \sum_{1\le|\alpha|\le r} \int\int_{E_0} \frac{h_{\mathcal{S}}^{2|\alpha|}}{|E_0|} (\mathcal{D}^\alpha Q(x,y))^2 dxdy, \tag{14}$$

where E_0 can be any cell $E \in \mathcal{S}$, and is usually referred to as the target cell, i.e., the cell E form witch (x, y) is taken in $Q^{\mathcal{S}}_{r,r}(x, y)$.

The Jiang-Shu smoothness indicator can be computed utilizing the predefined basis polynomials (11):

$$\begin{aligned}
\sigma_{JS} &= \sum_{1\le|\alpha|\le r} \frac{h_{\mathcal{S}}^{2|\alpha|}}{|E_0|} \iint_{E_0} (\mathcal{D}^\alpha Q(x,y))^2 dxdy \\
&= \frac{h_{\mathcal{S}}^2}{|E_0|} \sum_{1\le|\alpha|\le r} \iint_{\hat{E}_0} (\mathcal{D}^\alpha \hat{Q}(\xi,\eta))^2 d\xi d\eta \\
&= \frac{h_{\mathcal{S}}^2}{|E_0|} \sum_{1\le|\alpha|\le r} \iint_{\hat{E}_0} \big(\sum_{ij} \overline{u}_{ij} \mathcal{D}^\alpha \hat{\phi}_{i,j}(\xi,\eta)\big)^2 d\xi d\eta \\
&= \frac{h_{\mathcal{S}}^2}{|E_0|} \sum_{ij}\sum_{i'j'} \overline{u}_{ij}\overline{u}_{i'j'} \sum_{1\le|\alpha|\le r} \iint_{\hat{E}_{ij}} \mathcal{D}^\alpha \hat{\phi}_{ij}(\xi,\eta) \mathcal{D}^\alpha \hat{\phi}_{i'j'}(\xi,\eta) d\xi d\eta \\
&= \frac{h_{\mathcal{S}}^2}{|E_0|} \sum_{ij}\sum_{i'j'} \overline{u}_{ij}\overline{u}_{i'j'} \sigma_{ij}^{i'j'},
\end{aligned} \tag{15}$$

where the coefficient tensor $\sigma_{ij}^{i'j'}$ can be precomputed before the time-stepping loop, after the mesh is established.

Alternatively, we can compute the Jiang-Shu smoothness indicator directly using the polynomial coefficients formulation (9) as

$$\begin{aligned}
\sigma_{JS} &= \frac{h_{\mathcal{S}}^2}{|E_0|} \sum_{1\le|\alpha|\le r} \iint_{\hat{E}_0} (\mathcal{D}^\alpha \hat{Q}(\xi,\eta))^2 d\xi d\eta \\
&= \sum_{|\gamma|\le r}\sum_{|\beta|\le r} c_\gamma c_\beta \Bigg(\sum_{1\le|\alpha|\le r} \frac{h_{\mathcal{S}}^2}{|E_0|} \iint_{E_0} \mathcal{D}^\alpha \xi^\gamma \mathcal{D}^\alpha \eta^\beta \, d\xi d\eta \Bigg) \\
&= \sum_{|\gamma|\le r}\sum_{|\beta|\le r} c_\gamma c_\beta \zeta_{\gamma,\beta},
\end{aligned} \tag{16}$$

which paves the way for further simplification in (19) below.

The smoothness indicator exhibits the following convergence property: there exists a constant $C > 0$ such that when $h \to 0^+$,

$$\sigma = \begin{cases} Ch^2 + \mathcal{O}(h^3) & \text{u is smooth on } \Omega_{\mathcal{S}}, \\ \Theta(1) & \text{u is not smooth on } \Omega_{\mathcal{S}}. \end{cases} \tag{17}$$

Actually $\sigma = O(1)$ in the nonsmooth case, but in practice we find that $\sigma = \Theta(1)$. Being $\Theta(1)$ means that σ is bounded above and velow by a constant.

0.2.3 Two Alternative Smoothness Indicators

We propose two alternative ways of defining a smoothness indicator

- *Simple smoothness indicator* Huang et al. (2023)

$$\sigma_{\text{simp}} = \frac{1}{N_{\mathcal{S}} - 1} \sum_k (\overline{u}_k - \overline{u}_0)^2, \tag{18}$$

  where $\overline{u}_0$ is the cell averaged solution in the target cell. This simple smoothness indicator is designed to reduce the computational cost and to deal with distorted stencils on unstructured meshes. However, this simple smoothness indicator is not as robust as the classic Jiang-Shu smoothness indicator.

- *Polynomial smoothness indicator* Arbogast et al. (2025)

$$\sigma_P = \sum_\alpha \zeta_{\alpha,\alpha} c_\alpha^2 \tag{19}$$

  which is obtained by dropping all the cross terms appeared in the formulation (16). This alternative serves as a good approximation to the classic Jiang-Shu smoothness indicator while the computing efficiency is greatly improved.

0.2.4 ML-WENO weighting

Given a target cell $E_{i,j}$, we can create a nonempty set of stencils $\{\mathcal{S}_l \ni E_{i,j}, l = 0, 1, \ldots, L, L \geq 0\}$.(See Figure 1 for an example.) The reconstruction of u on the target cell $E_{i,j}$ is a convex combination of the stencil polynomials defined in equation (12)

$$\mathcal{R}(\mathbf{x}) = \sum_l \tilde{\omega}_l Q^{\mathcal{S}_l}(\mathbf{x}), \quad \mathbf{x} \in E_{i,j}, \tag{20}$$

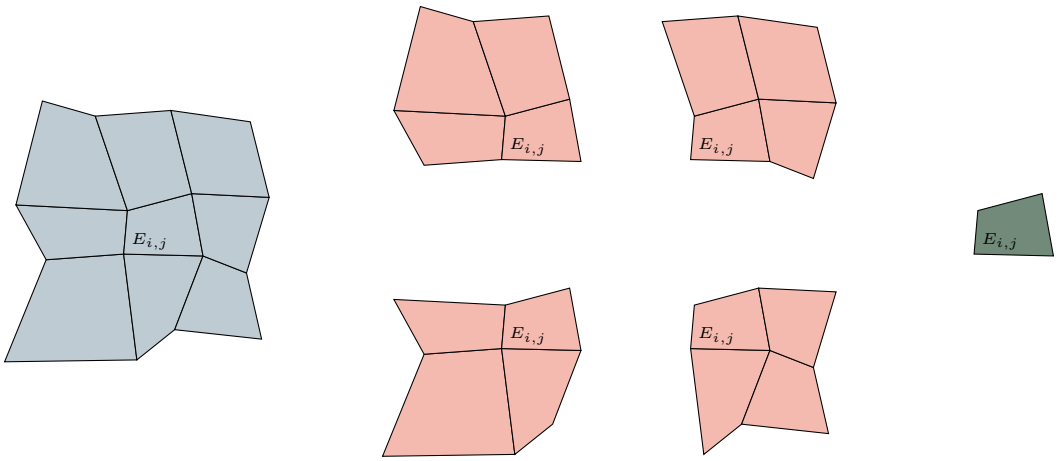


Figure 1: An example of a (3,2,1) multi-level stencils combination on a logically rectangular quadrilateral mesh.

where $\sum_l \tilde{\omega}_l = 1$ and $\tilde{\omega}_l$ are the normalized nonlinear weights associated with stencils. We begin by arbitrarily choosing a set of linear weights $\{\omega_l\}$ and scale them with stencil smoothness indicator as

$$\hat{\omega}_l = \frac{\omega_l}{(\sigma_l + \epsilon h_0^2)^{ar_l + n_l}}, \tag{21}$$

where $r_l = \max\{s, t\} = r$ in the cases of using square stencils, and the constants ϵ = 1e-4 and $a > 0.5$ are chosen accordingly. The biasing factor $n_l = n(r_l)$ is used to biasing further away from the constant level, whose smoothness indicator is always zero. We choose

$$n(r_l) = n_l = \begin{cases} 1, & \text{if} \quad r_l = 1, \\ 3, & \text{if} \quad r_l = 2, \\ 4, & \text{if} \quad r_l \geq 3. \end{cases} \tag{22}$$

We finally renormalise the scaled weights as

$$\tilde{\omega}_l = \frac{\hat{\omega}_l}{\sum_{l'} \hat{\omega}_{l'}}. \tag{23}$$

0.2.5 The ML-WENO Reconstruction Error

In this subsection, we analyze the accuracy of the new multi-level weighting scheme. Consider the set containing indices of stencils $l \in \mathcal{L} = \{0, 1, 2, \dots L\}$. Specifically, we denote the set of indices of stencils of size $s \times t$ as $\mathcal{L}_{s,t} \subset \mathcal{L}$. We assume the mesh resolution is sufficiently fine such that at least one smooth stencil exists in the

presence of shocks. Accordingly, we define two index sets: one containing the stencils identified as smooth, and the other containing those that exhibit discontinuities as

$$\begin{aligned} \text{Smooth} &= \{l : \ \ u \text{ is smooth on } \mathcal{S}_l\}, \\ \text{Jump} &= \{l : \ \ u \text{ has a jump on } \mathcal{S}_l\}, \end{aligned} \tag{24}$$

where Smooth$\cup$Jump $= \mathcal{L}$ and Smooth$\cap$Jump $= \emptyset$. Moreover, we assume Smooth $\neq \emptyset$ We show that the reconstruction is accurate with respect to the target cell E_0 to order

$$r_{\max} = \max\{r : \text{Smooth} \cap \mathcal{L}_{r,r} \neq \emptyset\}. \tag{25}$$

Figure 2 illustrates a (3,2,1) combination of stencils in the presence of two shocks. In this example, the set containing indices of smooth stencils is Smooth $= \{0, 1\}$, indicating that u on stencils $\mathcal{S}_0$ and $\mathcal{S}_1$ does not have a jump. Consequently, we expect the accuracy of the reconstruction is $r_{\max} = \text{Smooth} \cap \mathcal{L}_{2,2} = 2$.

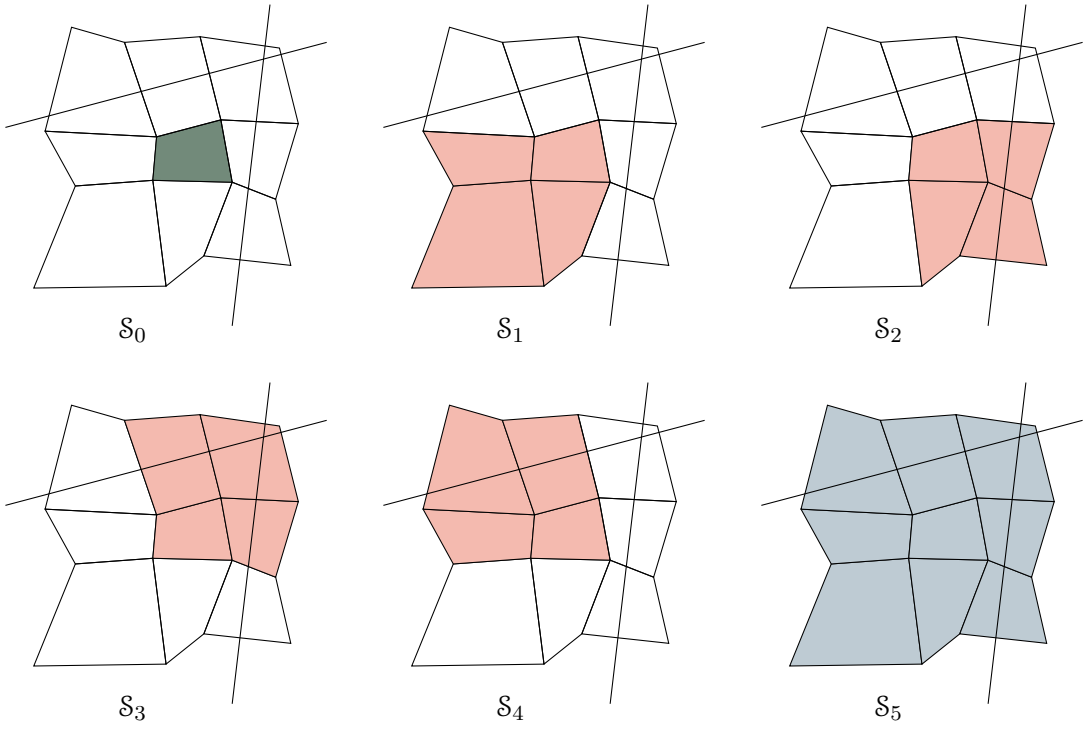


Figure 2: An illustration of a (3,2,1) multi-level stencils combination on a logically rectangular quadrilateral mesh with two shock present.

We follow the estimation of reconstruction error in Arbogast et al. (2024). We start by estimating the error of the nonlinear scaling (21) associated with some

stencils $\mathcal{S}_l$ for some $m > 0$.

$$\hat{\omega}_l = \frac{\omega_l}{(\sigma_l + \epsilon h_0^2)^m}. \tag{26}$$

We estimate the reconstruction error under two distinct situtaions:

- $l \in$ Smooth, the smoothness indicator $\sigma_l = Ch^2 + \mathcal{O}(h^3)$;
- $l \in$ Jump, the smoothness indicator $\sigma_l = \Theta(1)$.

We employ the asymptotic estimation result from Olver (1997). Assuming $n \geq m$, $a > 0$, and $b > 0$, then

$$\begin{aligned} \frac{a}{bh^m + \mathcal{O}(h^n)} &= \frac{ah^{-m}}{b}\big(1 + \mathcal{O}(h^{n-m})\big), \\ \frac{a}{bh^{-n} + \mathcal{O}(h^{-m})} &= \frac{ah^n}{b}\big(1 + \mathcal{O}(h^{n-m})\big). \end{aligned} \tag{27}$$

For the smooth stencil $\mathcal{S}_l$, $l \in$ Smooth

$$\begin{aligned} \hat{\omega}_l = \frac{\omega_l}{(\sigma_l + \epsilon h_0^2)^m} &= \frac{\omega_l}{((C+\epsilon)h_0^2 + \mathcal{O}(h^3))^m} \\ &= \omega_l \Big(\frac{h_0^{-2}}{C+\epsilon}(1 + \mathcal{O}(h))\Big)^m \\ &= \frac{\omega_l}{(D+\epsilon)^m} h_0^{-2m}(1 + \mathcal{O}(h)). \end{aligned} \tag{28}$$

In conclusion, we estimate the nonlinear scaling as

$$\hat{\omega}_l = \begin{cases} \dfrac{\omega_l}{(C+\epsilon)^m} h_0^{-2m}(1 + \mathcal{O}(h)), & \text{if} \quad l \in \text{Smooth}, \\ \Theta(1), & \text{if} \quad l \in \text{Jump}. \end{cases} \tag{29}$$

Then we estimate the error for the normalized nonlinear weighting (23). We expand the sum of nonlinear scaling as

$$\sum_l \hat{\omega}_l = \sum_r \sum_{\substack{l \in \\ \text{Smooth} \cap \mathcal{L}_{r,r}}} \hat{\omega}_l + \sum_{l \in \text{Jump}} \hat{\omega}_l. \tag{30}$$

We estimate the smooth part of the nonlinear weighting as

$$\sum_r \sum_{\substack{l \in \\ \text{Smooth} \cap \mathcal{L}_{r,r}}} \hat{\omega}_l = \sum_r \sum_{\substack{l \in \\ \text{Smooth} \cap \mathcal{L}_{r,r}}} \frac{\omega_l}{(C+\epsilon)^m} h_0^{-2(ar+\eta)}(1 + \mathcal{O}(h)). \tag{31}$$

Assume $h < 1$, we keep the largest $ar + \eta(r)$ number as our estimator

$$\begin{aligned}\sum_r \sum_{\substack{l\in\\ \text{Smooth}\cap\mathcal{L}_{r,r}}} \hat{\omega}_l &= \sum_{\substack{l\in\\ \text{Smooth}\cap\mathcal{L}_{r_{\max},r_{\max}}}} \frac{\omega_l}{(C+\epsilon)^{ar_{\max}+\eta(r_{\max})}} h_0^{-2(ar_{\max}+\eta(r_{\max}))}(1+\mathcal{O}(h)) \\ &\qquad + \Theta(h^{-2(ar+\eta(r))}) \\ &= \sum_{\substack{l\in\\ \text{Smooth}\cap\mathcal{L}_{r_{\max},r_{\max}}}} \frac{\omega_l}{(C+\epsilon)^{ar_{\max}+\eta(r_{\max})}} h_0^{-2(ar_{\max}+\eta(r_{\max}))} \\ &\qquad + \mathcal{O}(h^{1-2(ar_{\max}+\eta(r_{\max}))}).\end{aligned} \tag{32}$$

later, we denote the set $\text{Smooth}\cap\mathcal{L}_{r,r}$ as Smooth_r for simplicity. The overall accuracy estimation of the nonlinear weighting (23) is presented as

- $l \in \text{Smooth}$

$$\tilde{\omega}_l = \frac{\hat{\omega}_l}{\sum_{l'} \hat{\omega}_{l'}} = \frac{\omega_l}{\sum_{l'\in\text{Smooth}_r} \omega_{l'}} h^{2[a(r_{\max}-r)+\eta(r_{\max})-\eta(r)]}(1+\mathcal{O}(h)); \tag{33}$$

- $l \in \text{Jump}$

$$\tilde{\omega}_l = \frac{\hat{\omega}_l}{\sum_{l'} \hat{\omega}_{l'}} = \Theta(h^{2[a(r_{\max})+\eta(r_{\max})]}). \tag{34}$$

Now we can estimate the reconstruction error as

$$\begin{aligned}|u(\mathbf{x}) - \mathcal{R}(\mathbf{x})| &= |\sum_l \tilde{\omega}_l(u(\mathbf{x} - Q_l(\mathbf{x})))| \\ &\le \sum_{l\in\text{Smooth}} \tilde{\omega}_l |u(\mathbf{x}) - Q_l(\mathbf{x})| + \sum_{l\in\text{Jump}} \tilde{\omega}_l |u(\mathbf{x}) - Q_l(\mathbf{x})| \\ &\le \sum_r \sum_{l\in\text{Smooth}_r} \Theta(h^{2[a(r_{\max}-r)+(\eta(r_{\max})-\eta(r))]})\mathcal{O}(h^r) \\ &\qquad + \sum_{l\in\text{Jump}} \Theta(h^{2(ar_{\max}+\eta(r_{\max}))})\mathcal{O}(1) \\ &= \mathcal{O}(h^{r_{\max}})\end{aligned} \tag{35}$$

Combining everything above and (20), we arrive at the formal error estimate of the reconstruction.

Lemma 0.1. *Let $\mathcal{R}(\mathbf{x})$ be the reconstruction. Assuming a qusi-uniform mesh, $h_0 = \Theta(h)$, there exists a constant $C > 0$ such that for any $\mathbf{x} \in E_0$,*

$$|u(\mathbf{x}) - \mathcal{R}(\mathbf{x})| \le Ch^{r_{\max}}, \tag{36}$$

where $r_{\max}$ is defined by (25).

0.2.6 Time stepping

In this subsection, we briefly discuss some suitable time integration techniques. We focus on explicit schemes for two main reasons.

- The phase package cannot be accurately coupled over long time intervals;
- The Jacobian is difficult to evaluate due to the involvement of an MFEM solver in velocity computation.

While forward Euler method is sufficient for stability analysis, it only provides first-order accuracy. Therefore, we employ the strong-stability- preservation (SSP) scheme introduced in Gottlieb et al. (2001). This scheme achieves higher order time accuracy while theoretically preserving the stability properties of the forward Euler method.

To begin with, the first-order forward Euler method is written as

$$u^{n+1} = u^n + \Delta t \tilde{\mathbf{F}}(u^n), \tag{37}$$

where $\tilde{\mathbf{F}}(u^n)$ represents the numerical integration of the numerical flux $F(u^n)$ in Equation (6). We then state a multi-stage explicit SSP Runge-Kutta method as

$$\begin{aligned} u^0 &= u^n, \\ u^{(i)} &= \sum_{j=0}^{i-1} \alpha_{i,j} u^{(j)} + \Delta t \beta_{i,j} \tilde{\mathbf{F}}(u^{(j)}), \quad 1 \le i \le m \\ u^{n+1} &= u^m, \end{aligned} \tag{38}$$

where $\sum_j^{i-1} \alpha_{i,j} = 1$ for consistency. In consistent with the MFEM solver's second-order accuracy in velocity, we omit further discussion of time-integration schemes beyond third-order accuracy. The second- and third-order explicit SSP Runge-Kutta schemes can be expressed in the multi-stage form (38) as

$$\begin{aligned} u^0 &= u^n \\ u^{(1)} &= u^{(0)} + \Delta t \tilde{\mathbf{F}}(u^{(0)}) \\ u^{n+1} &= u^{(0)} + \frac{1}{2}\Delta t \tilde{\mathbf{F}}(u^{(0)}) + \frac{1}{2}\Delta t \tilde{\mathbf{F}}(u^{(1)}), \end{aligned} \tag{39}$$

and

$$\begin{aligned} u^0 &= u^n, \\ u^{(1)} &= u^{(0)} + \Delta t \tilde{\mathbf{F}}(u^{(0)}), \\ u^{(2)} &= \frac{3}{4}u^n + \frac{1}{4}u^{(1)} + \frac{1}{4}\Delta t \tilde{\mathbf{F}}(u^{(1)}) \\ u^{n+1} &= \frac{1}{3}u^{(0)} + \frac{2}{3}u^{(2)} + \frac{2}{3}\Delta t \tilde{\mathbf{F}}(u^{(2)}). \end{aligned} \tag{40}$$

0.3 Handling Boundary Conditions

We couple MFEM and FVM solver on a single mesh in our simulation. Therefore, accurate and compatible boundary conditions are assigned accordingly. However, it is usually difficult to manage boundary conditions for higher order finite volume and finite difference method, which require wide stencils.

Some methods address this difficulty by using ghost cells or ghost points outside the computational domain, allowing the use of interior-like stencils near boundaries. However, the extrapolation or estimation of the manufactured values of ghost cells or ghost points are not straightforward. A notable method, the inverse Lax-Wendroff procedure, iteratively substitute normal derivatives of the solution with tangential and time derivatives using the PDE relation to impose more precise values for the ghost cells or ghost points.

We focus on utilizing the flexibility of the ML-WENO reconstruction to tackle the difficulties in a direct way without the use of ghost cells. This approach is independent of the complexity of the governing partial differential equations and intricacies of the geometry of the boundary. Similar discussion using the adaptive order reconstruction can be found in Semplice and Visconti (2020) and Semplice et al. (2023).

In Finite Volume Methods, we must use the boundary condition to determine the numerical flux on the boundary. To assign proper boundary conditions to a FVM discretization, it is necessary to first identify whether the flow is inflow or outflow first. Let u_B denote the value of u on the boundary. We classify the types of boundary conditions based on the magnitude and direction of the numerical flux $\mathbf{f}(u_B) \cdot \boldsymbol{\nu}$.

- If $\mathbf{f}(u_B) \cdot \boldsymbol{\nu} > 0$, it is classified as an *outflow* boundary, and no fixed boundary condition should be assigned. Instead a *free outflow* boundary condition is used, where the numerical flux is simply given by $\mathbf{f}(u_B) \cdot \boldsymbol{\nu}$. The outflow boundary may lead to degradation in the accuracy of the numerical solution, as high-order stencils can extend beyond the computational domain. ML-WENO offers the flexibility to add stencils of the desired size that span back into the domain, thereby preserving accuracy near outflow boundaries.

- If $\mathbf{f}(u_B) \cdot \boldsymbol{\nu} < 0$, it is classified as an *inflow* boundary. We can understand it as prescribing the inflow flux $\mathbf{f}_B$ or fixing the boundary value u_B as an equivalent to Dirichlet boundary in the context of Finite Element Method. In the inflow case, discontinuous shock may enter the domain, requiring a swift response inside the domain to align with the specified Dirichlet value. To accommodate such potential discontinuities, we can incorporate an extra constant stencil polynomial to the inflow boundary.

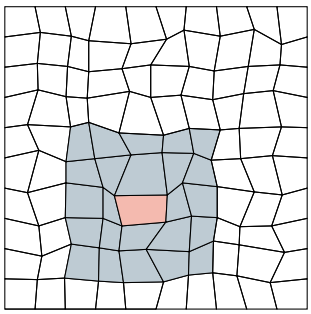
- If $\mathbf{f}(u_B) \cdot \boldsymbol{\nu} = 0$, it is usually referred to as a *noflow* boundary. No additional treatment on the boundary is required, as the value is fixed at zero.

In the following subsections, we present two examples: the first illustrates how additional higher order stencils affect reconstruction order, while the second address an inflow Dirichlet boundary condition that generates an incoming shock wave.

0.3.1 Additional stencils reinforced at the boundary

Consider a standard (5,3) reconstruction on a $[0,1]^2$ domain with the function

$$u(x,y) = \begin{cases} \sin(x)\cos(y) & x \le 0.5, \\ \sin\left(\dfrac{(x+0.3)\pi}{2}\right)^6 \cos\left(\dfrac{(y-0.6)\pi}{2}\right)^6 + 0.5 & x > 0.5. \end{cases} \tag{41}$$



0.3.2 Inflow at the boundary

For the inflow example, we use the Burgers-like equation $u_t + u^3 u_x = 0$ with the flux $\mathbf{f}(u) = (u^4/4, 0)$ in the domain $(0,1)^2$. The Dirichlet boundary condition on the left boundary

$$u_D(0,y,t) = \begin{cases} \sin^2(2\pi y), & y \le 0.25, y \ge 0.75, \\ 1, & 0.25 < y < 0.75. \end{cases} \tag{42}$$

0.4 Numerical Test

We test our ML-WENO algorithm with Riemann shock and rarefaction. Let $\mathbf{f}(u) = (u^2/2, 0)$ be the Burgers flux function. In this example, we take the initial

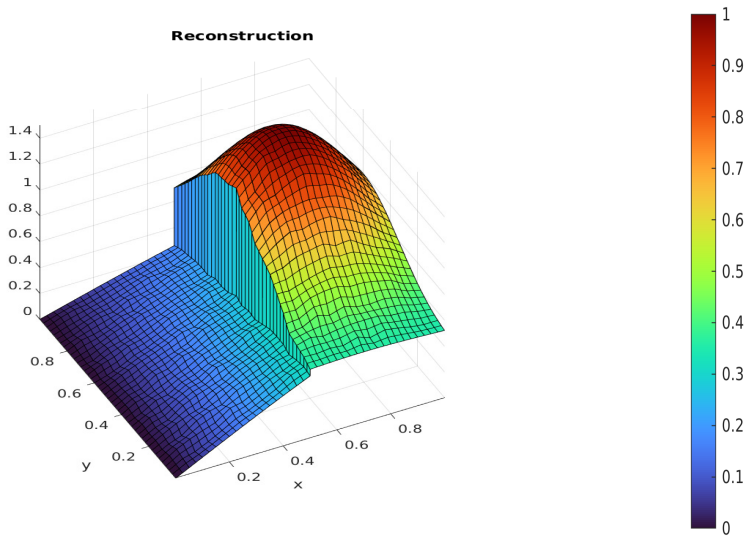


Figure 3: The reconstructed solution on the 40×40 randomly distorted logically rectangular mesh.

condition $u_0(x, y) = 1$ in the strip $0.5 < x < 1.5$ and 0 elsewhere within the computational domain $\Omega = [0, 3] \times [0, 1]$. We assign zero flux boundary condition on the two sides $y = 0$ and $y = 1$.

The true solution is

$$u(x, y, t) = \begin{cases} 0, & x < 0.5, x \geq 0.5t + 1.5, \\ (x - 0.5)/t, & 0.5 \leq x < t + 0.5, \\ 1, & t + 0.5 \leq x < 0.5t + 1.5. \end{cases} \tag{43}$$

We forward marching to the time = 2.0, when trailing rarefaction reaches the leading shock. For simplification, we test the problem on a 180×60 rectangular mesh. We compute the solution to time 2 using the standard SSP3 Runge-Kutta time stepping scheme, with time step $\Delta t = 0.2h$.

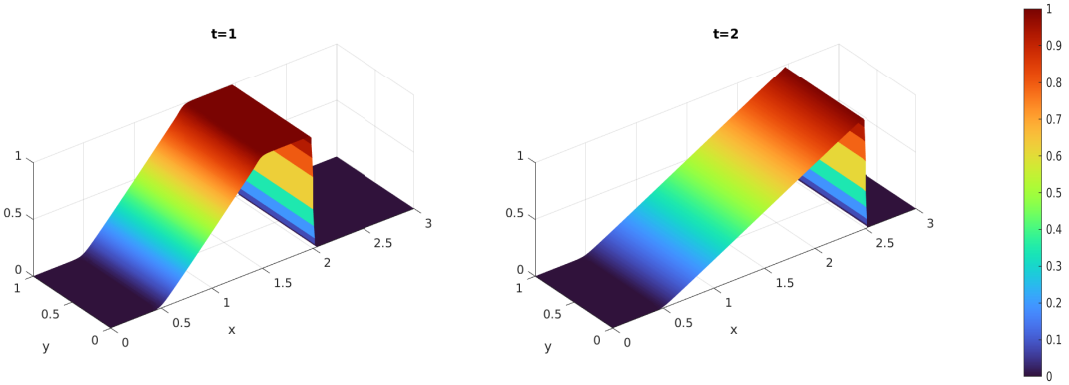


Figure 4: The reconstructed solution on the 180×60 rectangular mesh at times 1.0 and 2.0. (The rare faction and shock meet at time 2.0.)

Chapter 1: Code Implementation

In this project, all numerical experiments and simulations are implemented in C++/C, with basic iterative solvers and simple parallel data management facilitated by PETSc Balay et al. (2025). In this section, we describe the implementation of a coupled FEM-FVM code, designed to preserve generality and support future optimization and extensibility.

We first present a general implementation for an unstructured mesh, followed by a detailed discussion of a specfic application involving a logically rectangular mesh. All applications are considered in a two-dimensional setting.

1.1 Geometry

A fundamental step in developing a FEM or FVM code is the indexing of geometric entities. The mesh index data structure implemented in the code is inspired by the classical viewpoint of Boundary Representation (BREP), originally introduced by Baumgart (1975). In the two-dimensional case, each element is bounded by a set of edges, and each edge is defined by a set of vertices. We begin by introducing the following notations for global geometric objects:

$$\begin{aligned} \text{Vertex} \quad & V_i, \quad i \in ID_V, \\ \text{Edge} \quad & E_i, \quad i \in ID_E, \\ \text{Element (i.e. Face)} \quad & F_i, \quad i \in ID_F, \end{aligned} \tag{1.1}$$

where ID_V, ID_E and ID_F denote countable sets containing all unique global indices for vertices, edges, and elements (or cells, i.e., the faces), respectively. Superscripts are used to indicate subsets of these global index sets. For example, $ID_V^{E_i} \subset ID_V$ represents the set of global vertex indices associated with edge E_i, which is a subset of the global vertex index set.

We again present the following notations for local geometric entities:

$$\begin{aligned} \text{Vertex} \quad & v_i, \quad i \in id_v, \\ \text{Edge} \quad & e_i, \quad i \in id_e, \\ \text{Element (i.e. Face)} \quad & f_i, \quad i \in id_f, \end{aligned} \tag{1.2}$$

where id_v, id_e and id_c denote countable sets containing local index for vertices, edges and elements corresponding to a given geometrical object. Superscripts are used to indicate the target geometrical object. For example, $id_v^{E_i} = \{0, 1\}$ represents the local index of the two vertices bounding the edge E_i.

We establish a one-to-one mapping between local and global indices through a local-to-global map, defined as

$$f : id \to ID. \tag{1.3}$$

In practice, it is often unnecessary to explicitly define a local-to-global mapping. On the other hand, for each local array containing global indices, the array indices naturally represent the local index set. This array-based representation enables the implicit construction of a global-to-local mapping within each geometric entity.

1.1.1 General Unstructured Mesh

In both FEM and FVM solvers, an element object F_i (mesh cell or "faces") serves as the fundamental geometry entity. These elements will be assumed to be strictly convex polygons, tessellating space , and sharing complete edges. we start with a brief analysis of the information each face should access from its local perspective. Each element F_i should have the access to the following geometric objects:

- The adjacent elements connected by edges;
- The adjacent elements connected with vertices, but not sharing an edge;
- The vertices of the polygonal element;

- The edges that bound this element.

At the same time, each edge E_j should have access to:

- Two adjacent elements that share this edge;
- Two vertices that bound this edge.

For simplicity, we restrict our discussion to quadrilaterals in the following discussions. In FEM and FVM computations, we assume an orientable mesh, where "clockwise" and "counterclockwise" orientations can be consistently defined. This allows the use of a half-edge data structure to systematically manage edge directions.

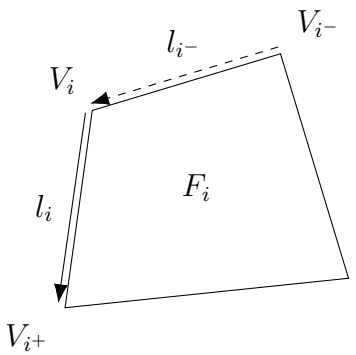


Figure 1.1: An illustration of the half-edge data structure for a quadrilateral.

Figure 1.1 illustrates a simple half-edge data structure applied to a quadrilateral mesh. For each element F_i, the associated vertex V_i references the outgoing link l_i that points to the next vertex V_{i^+}, while the incoming link l_{i^-}, which points to V_i is not directly referenced. Consequently, a normal "direction" can be determined from the perspective of each element object F_i in accordance to the direction of links.

1.2 Logically Rectangular Mesh

Mesh generation is not the primary focus of this project; therefore, we restrict to a logically rectangular mesh and adopt a simple approach to generating a

quadrilateral mesh. The quadrilateral mesh is generated by randomly distorting each interior vertex coordinates of a uniform rectangular mesh. However, the indexing of geometrical entities remain unchanged despite the vertex perturbation.

In our implementation, all indices are represented using Cartesian coordinates. This allows the local-to-global mapping of geometric entities to be established easily through linear functions. All geometric entities can be indexed using Cartesian coordinates straightforwardly. For a mesh with $M \times N$ cells, the total number of vertices is $(M+1) \times (N+1)$, while the total number of edges is $M \times (N+1) + (M+1) \times N$. Horizontal edges are indexed first, and vertical edges are indexed sequentially starting from the offset $M \times (N+1)$.

Before we delve into the indexing conventions of this type of mesh, we introduce a reshape function that facilitates communication between N-dimensional data and its one-dimensional representation.

1.2.1 Reshape of N-dimensional Data

The natural representation of a 2D Cartesian index is a pair (i, j). In practice, it is often reshaped into a 1D array for more efficient computational handling. We introduce a general reshape function that projects N-dimensional data to a one-dimensional data. This reshape function can also be used for tensor data, e.g., The Jiang-Shu smoothness indicators (15).

Consider a bijective map between N-dimensional data and its one-dimensional data representation as

$$i \leftrightarrow (i_0, i_1, \ldots, i_{N-1}). \tag{1.4}$$

Algorithm 1 illustrates a simple implementation of the reshape function. The inverse operation, reshaping one-dimensional data back into N-dimensional form, can be achieved using modulo and integer division operations.

Throughout this work, we use these two index representations interchangeably.

Algorithm 1 Reshape N-dimensional data

Consider a N-dimensional array of size $\{n_0 \times n_i \times \dots n_{N-1}\}$.

Let $I = 0$ be the variable storing the output, and $a = 1$ be the intermediate multiplier.

Consider an input N-dimensional index $\{i_0, i_1, \dots, i_{N-1}\}$.

1: **for** rank $r = 0, 1, \dots, N-1$ **do**

2: $\quad I = I + i_r \times a.$

3: $\quad a = a \times n_r.$

4: **end for**

To emphasize the relation between N-dimensional and its one-dimensional representation, we introduce the following notation as

$$i(i_0, i_1, \dots, i_{N_1})_{n_0, n_1, \dots, n_{N-1}}, \tag{1.5}$$

where i denotes the mapping from the N-dimensional data to the one-dimensional data, while $(i_0, i_1, \dots, i_{N-1})$ is the corresponding N-dimensional index. The N-dimensional quantities $n_0, n_1, \dots, n_{N-1}$ represent the respective sizes along each dimension of the N-dimensional array.

1.2.2 Local Vertex Representation

We now represent the local vertex index with the new notation (1.5). For any vertex, we can represent its local index as

$$a(a_0, a_1)_{2,2}, \tag{1.6}$$

where $a_0, a_1 \in \{0, 1\}$. We can link to the next point simply by adding 1 to the one-dimensional index under the understanding of modulo as

$$b \equiv a + 1 \pmod 4, \tag{1.7}$$

which is a simple half-edge data representation. The direction is naturally defined by this addition operation.

1.2.3 Local to Global Index Convention

Recall that local vertices and edges are indexed starting from bottom left corner, as illustrated in Figure 1.4. We now define the local to global index mapping with the help of the new notation (1.5). Consider any local vertex v_i associated to a quadrilateral element $F_{I(I_0,I_1)_{M,N}}$ with the local index

$$id_v \ni i(i_0, i_1)_{2,2}. \tag{1.8}$$

We find the corresponding global index with ease as

$$ID_V \ni I(I_0 + i_0, I_1 + i_1)_{M+1,N+1}. \tag{1.9}$$

We continue our discussion the local-to-global mapping for vertical and horizontal edges. Consider edge e_i defined by the vertex $id_v \ni i(i_0, i_1)$ and its half-edge link l_i:

- Global index of the horizontal edge is

$$ID_E \ni I(I_0, I_1 + i_1)_{M,N+1}; \tag{1.10}$$

- Global index of the vertical edge is

$$ID_E \ni I(I_0 + i_0, I_1)_{M+1,N} + M \times (N + 1). \tag{1.11}$$

1.3 Dofs and MFEM assembly

In FEM and FVM computations, it is necessary to associate geometrical entities with degrees of freedom (dofs). For FVM schemes, the dofs associated with edge fluxes are straightforward. However, in MFEM, the treatment of dofs requires more carefully handling, particularly for those located on the boundary. Following the approach of Hughes (1987), we employ three data processing arrays: ID, IEN and LM arrays. We begin by the introduction of these three arrays as:

- IEN array,

$$\mathrm{IEN}(a, e) = A, \tag{1.12}$$

where a is the local dof index, e is the global element number and A is the global dof index;

- ID array,

$$\mathrm{ID}(A) = \begin{cases} P, & A \notin \text{Essen}, \\ 0, & A \in \text{Essen}, \end{cases} \tag{1.13}$$

where P is the position used in global matrix assembly, and Essen denotes the boundary assigned with essential boundary condition;

- LM array,

$$\mathrm{LM} = \mathrm{ID}(\mathrm{IEN}(a, e)). \tag{1.14}$$

In actual code implementation, we create LM array by screening all dofs prior to computation, while IEN and ID array are not stored explicitly.

1.3.1 MFEM Indexing Example

For a first order $H(\mathrm{div})$ conforming element, the velocity dofs are primarily associated with edges, similar to FVM, making them relatively intuitive to understand and implement. Therefore, we provide a detailed description of how the velocity dofs of a modified Bernardi-Raugel element is implemented using ID, IEN, and LM arrays, where both vertex dofs and edge dofs are present.

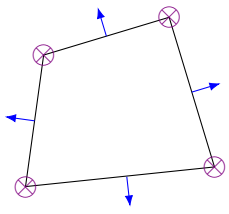


Figure 1.2: An exhibition of velocity dofs associated with a modified Bernardi-Raugel element.

Figure 1.2 illustrates the dof layout in a Bernardi-Raugel element: two dofs (the x-and y-components) are associated with each vertex node while one normal flux dof is associated with each edge node. The global dofs are ordered as follows: x-and y-component of the vertex dofs, followed by flux dofs on horizontal edges and finally flux dofs on vertical edges.

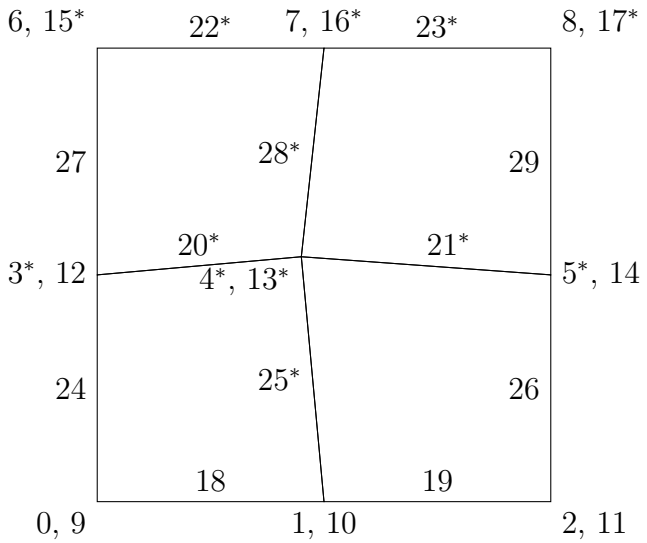


Figure 1.3: An exhibition of velocity dofs associated with Bernardi-Raugel on a 4×4 mesh.

Figure 1.3 shows a simple 4×4 mesh with corresponding index associated with each velocity dof marked. The total dof counts equal to $2 \times (3 \times 3) + 2 \times 3 + 3 \times 2 = 30$. For example, we assume the following boundary conditions are prescribed.

- Bottom Edge:
  - x-component is fixed with zero;
  - y-component is fixed with a constant value;
  - normal flux is fixed with a constant value.
- Left and Right side Edges:

  - x-component is fixed with zero;
  - y-component is free, given natural boundary condition (zero stress);
  - normal flux is fixed with a zero flux.

- Top Edge:
  - x-component fixed with zero;
  - y-component is free, given natural boundary condition (zero stress);
  - normal flux is free.

In Figure 1.3, we mark all the dofs that are not prescribed, i.e., will be determined by solving the linear system, with a superscript $*$. These *free* dofs consist of interior dofs as well as boundary dofs subject to natural boundary conditions. We can construct an ID array accordingly in Table 1.1. As a result, the reduced linear

P	0	1	2	3	4	5	6	7	8	9	10	11
A	3	4	4	13	14	16	20	21	22	23	25	28

Table 1.1: ID array generated to the dofs in Figure 1.3.

system excluding prescribed velocity dofs is 12×12, which is the size of the ID array in Table 1.1.

1.4 Parallel Implementation

In this section, we disscuss some ideas regarding parallel implementation of MFEM and FVM solvers, and present some preliminary timing results for each solver. The parallel implementation of the MFEM and FVM need different treatments.

For FVM discretization, the high order stencils posed some difficiulties on parallel implementation. We need to extend from one processor to the adjecent processor with the so-called ghost region, which results in more communication during computation. Consider a ML-WENO(3,2) scheme, as shown in Figure 1.4, we show

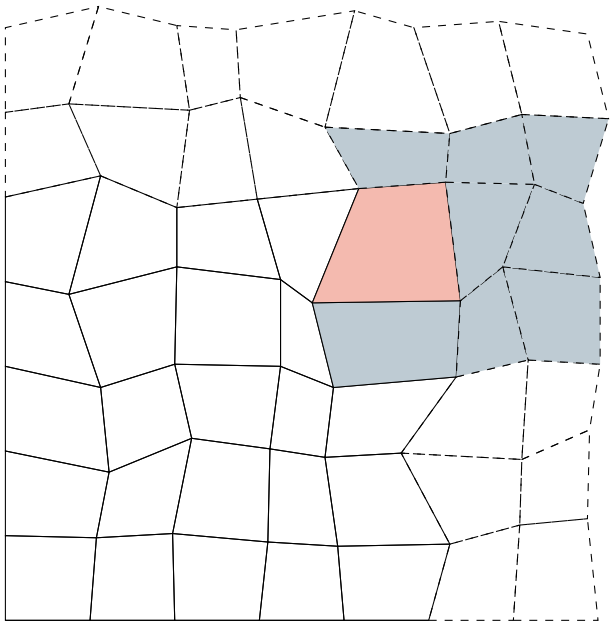


Figure 1.4: An exhibition of ghost region in parallel implementation of ML-WENO scheme.

distributed ghost region with dashed lines. We need to determine the flux acorss the edge of the red cell. Therefore, a maximum 3×3 stencil should be attached to the outside cell. Hence, in a ML-WENO(3,2) scheme, a two-cell ghost layer should be distributed and updated everytime computing flux.

For MFEM discretization, we encounter different problems. The MFEM discretization, unlike FVM scheme, forms a sparse matrix. The sparse matrix are splited across the processors. The dof in solution vector should be distributed accordingly. Therefore, everytime the solution is being reconstructed, communication between processors will be induced.

We provide strong scaling plot for our test parallel running. We run our MFEM and FVM code with 1,2,4,6 processors

1.4.1 Parallel Index Convention

Index convention in parallel computing requires another "local" index kept within each processor. We can bypass such difficultity by using Hash table data structure that directly mapping to global index with out adding another "local" index. However, in the worst case Hash table suffers $O(n)$ time complexity in searching.

1.5 PETSc Implementation

PETSc, the Protable, Extensible Toolkit for Scientific Computation has been gaining popularity in scientific applications modelled by partial differential equations **?** and Balay et al. (2025). We used three main features from PETSc package.

- Vector (Vec) and matrix (Mat) data structure.
- Data management object (DM).
- Standard linear solver (KSP).

We store global data in Vec and Mat data structure provided by PETSc. Vec and Mat objects in PETSc can complete simple add, dot operations without distinguish between sequential or parallel.

Note that PETSc provides a built-in data management object DMStag. However, we use a more generic data management object DMDA for structured mesh because we formed reduced linear system in MFEM computation. DMStag, on the other hand, does not provide a straightforward function that can create reduced system for us.

We use the built in KSP solver, e.g., KSPMINRES and KSPCG solver for our application when we solve large sparse system. However, for small system such as 4×4 system solved in ML-WENO stencil polynomial computation, we call lapack directly. We also use provided preconditioner.

Works Cited

Todd Arbogast, Chieh-Sen Huang, and Chenyu Tian. A finite volume multilevel weno scheme for multidimensional scalar conservation laws. *Comput. Methods Appl. Mech. Engrg.*, 421, 2024.

Todd Arbogast, Chieh-Sen Huang, Chenyu Tian, and Gabirel M.Gray. An efficient and accurate approximation to the classic polynomial smoothness indicator. *Submitted.*, 2025.

Satish Balay, Shrirang Abhyankar, Mark F. Adams, Steven Benson, Jed Brown, Peter Brune, Kris Buschelman, Emil Constantinescu, Lisandro Dalcin, Alp Dener, Victor Eijkhout, Jacob Faibussowitsch, William D. Gropp, Václav Hapla, Tobin Isaac, Pierre Jolivet, Dmitry Karpeev, Dinesh Kaushik, Matthew G. Knepley, Fande Kong, Scott Kruger, Dave A. May, Lois Curfman McInnes, Richard Tran Mills, Lawrence Mitchell, Todd Munson, Jose E. Roman, Karl Rupp, Patrick Sanan, Jason Sarich, Barry F. Smith, Hansol Suh, Stefano Zampini, Hong Zhang, Hong Zhang, and Junchao Zhang. PETSc/TAO users manual. Technical Report ANL-21/39 - Revision 3.23, Argonne National Laboratory, 2025.

Bruce G. Baumgart. A polyhedron representation for computer vision. In *Proceedings of the May 19-22, 1975, National Computer Conference and Exposition*, AFIPS '75, page 589–596, New York, NY, USA, 1975. Association for Computing Machinery. ISBN 9781450379199. doi: 10.1145/1499949.1500071. URL `https://doi.org/10.1145/1499949.1500071`.

Todd Dupont and Ridgway Scott. Polynomial approximation of functions in Sobolev spaces. *Mathematics of Computation*, 34(150):441–463, 1980.

Oliver Friedrich. Weighted essentially non-oscillatory schemes for the interpolation of mean values on unstructured grids. *Journal of Computational Physics*,

144(1):194–212, 1998. ISSN 0021-9991. doi: https://doi.org/10.1006/jcph.1998.5988. URL https://www.sciencedirect.com/science/article/pii/S0021999198959885.

Sigal Gottlieb, Chi-Wang Shu, and Eitan Tadmor. Strong stability-preserving high-order time discretization methods. *SIAM Review*, 43(1):89–112, 2001. doi: 10.1137/S003614450036757X. URL https://doi.org/10.1137/S003614450036757X.

Chieh-Sen Huang, Todd Arbogast, and Chenyu Tian. Multidimensional weno-ao reconstructions using a simplified smoothness indicator and applications to conservation laws. *J. Sci. Comput.*, 97, 2023.

Thomas J.R. Hughes. *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*. Dover Publications, 1987. ISBN 9780486411811.

Guang-Shan Jiang and Chi-Wang Shu. Efficient implementation of weighted eno schemes. *J. Comput. Phys.*, 126:202–228, 1996.

Frank Olver. *Asymptotics and special functions*. AK Peters/CRC Press, 1997.

M. Semplice and G. Visconti. Efficient implementation of adaptive order reconstructions. *Journal of Scientific Computing*, 83, 2020.

M. Semplice, E. Travaglia, and G. Puppo. One-and multi-dimensional cwenoz reconstructions for implementing boundary conditions without ghost cells. *Communications on Applied Mathematics and Computation*, 5:143–169, 2023.